

Rapport du Projet

Gestionnaire de QCM – Questions pour un Tekien

24/05/2025 – Module : Informatique 2

Réalisé par :

- Najm-Eddine ASABBAN
- Sekou SOUMAH

SOMMAIRE

- 1. Introduction
 - 1.1 Description de l'équipe
 - 1.2 Description du sujet
- 2. Organisation du projet et flux de travail
 - 2.1 Organisation de l'équipe
 - 2.2 Flux de travail
- 3. Problèmes rencontrés et solutions apportées
- 4. Résultats obtenus
 - 4.1 Mode enseignant
 - 4.2 Mode étudiant
 - 4.3 Gestion par catégories
- 5. Conclusion

1. Introduction

1.1 Description de l'équipe

Ce projet a été réalisé en binôme par Najm-Eddine ASABBAN et Sekou SOUMAH, tous les deux en préING1 à CY Tech. On se connaissait déjà du groupe de TD ce qui a facilité la communication dès le départ. On a essayé de se répartir les tâches de façon équilibrée en fonction de ce que chacun maîtrisait le mieux, même si on a souvent travaillé ensemble sur les parties les plus compliquées.

1.2 Description du sujet

Le sujet qu'on a choisi est le projet QCM, aussi appelé Questions pour un Tekien. L'idée c'est de remplacer Moodle qui était supposément en panne, en développant une application en langage C qui permet aux enseignants de créer et sauvegarder des QCM paramétrables, et aux étudiants de les passer directement dans le terminal et d'obtenir leur note automatiquement sur 20.

L'application devait proposer deux modes distincts : un mode enseignant protégé par mot de passe pour créer les QCM, et un mode étudiant pour les passer. Les QCM devaient être enregistrés dans des fichiers texte pour pouvoir être réutilisés à tout moment.

2. Organisation du projet et flux de travail

2.1 Organisation de l'équipe

Pour ce projet on a vraiment beaucoup utilisé Discord. C'était notre outil principal pour tout : on s'envoyait les fichiers .c et .h quand on travaillait chacun de son côté, on faisait des appels vocaux pour coder ensemble en temps réel, et on se partageait les erreurs de compilation pour que l'autre puisse aider à déboguer. Sans Discord on aurait perdu beaucoup de temps.

On a aussi utilisé un dépôt Git sur GitHub pour centraliser le code. Même si au début on n'était pas très à l'aise avec Git, on a vite pris le réflexe de commit après chaque avancée importante. Ça nous a sauvés plus d'une fois quand on avait cassé quelque chose et qu'on voulait revenir en arrière.

La répartition des tâches s'est faite un peu des deux façons : certaines fonctions ont été développées individuellement, comme la fonction `creer_qcm()` d'un côté et `mode_etudiant()` de l'autre, et d'autres parties comme la gestion des fichiers et les sécurités de saisie ont été faites ensemble en appel Discord.

2.2 Flux de travail

On a travaillé de façon progressive. On a d'abord mis en place la structure du projet : les fichiers `fichier.h`, `main.c`, `fonction.c` et le Makefile. Ensuite on a développé le menu principal, puis le mode enseignant avec la création et l'enregistrement des QCM, et enfin le mode étudiant avec le chargement des questions, le passage du QCM et le calcul de la note.

À chaque étape on testait ce qu'on venait de coder avant de passer à la suite. On s'envoyait les captures d'écran sur Discord pour valider que ça marchait bien des deux côtés. Quand il y avait un bug, on en discutait en appel et on cherchait ensemble.

3. Problèmes rencontrés et solutions apportées

On a rencontré plusieurs difficultés au cours du projet, certaines plus longues à résoudre que d'autres.

Une des difficultés qu'on a rencontrées c'était l'affichage ASCII du menu principal. On voulait faire quelque chose de propre avec des encadrés et des lettres stylisées, mais il suffisait d'un seul espace en trop ou en moins pour que tout le cadre soit décalé. On a dû compter les caractères à la main et tester régulièrement dans le terminal pour que tout s'affiche correctement.

On a aussi eu des problèmes avec la gestion du buffer clavier en C. Après chaque `scanf()`, le caractère de retour à la ligne restait en mémoire et venait perturber les `fgets()` suivants, ce qui

faisait que certaines saisies étaient sautées sans qu'on comprenne pourquoi. On a mis du temps à identifier le problème, et la solution était de vider le buffer avec une boucle `while(getchar() != '\n')` avant chaque lecture de chaîne.

On a aussi eu du mal avec la lecture des fichiers QCM. Quand on chargeait les questions avec `fscanf()` puis `fgets()`, le `'\n'` laissé par `fscanf()` faisait lire une ligne vide à `fgets()`. Il fallait consommer ce caractère avec `fgetc()` avant chaque `fgets()`, ce qu'on a découvert après pas mal d'essais.

4. Résultats obtenus

4.1 Mode enseignant

Le mode enseignant est fonctionnel et protégé par mot de passe. Un professeur peut créer un nouveau QCM en lui donnant un nom, en choisissant sa catégorie (Informatique, Mathématiques ou Divers), en activant ou non les points négatifs et le mode séquentiel. Il peut aussi modifier le mot de passe depuis ce mode, avec une déconnexion automatique pour la sécurité. Les QCM créés sont sauvegardés dans des fichiers texte et référencés dans les fichiers de catégorie.

4.2 Mode étudiant

Le mode étudiant permet de choisir une catégorie, de voir la liste des QCM disponibles, puis de passer le QCM choisi question par question. Selon le paramétrage du QCM, l'étudiant peut passer une question ou est obligé de répondre avant de passer à la suivante. À la fin, la note est calculée automatiquement sur 20, avec prise en compte des points négatifs si activés, et affichée dans un encadré ASCII.

4.3 Gestion par catégories

On a implémenté le rangement des QCM par catégories comme amélioration. Chaque QCM créé est automatiquement ajouté au fichier de sa catégorie. Au moment de passer un QCM, l'étudiant choisit d'abord sa catégorie et ne voit que les QCM qui en font partie. Trois QCM sont fournis au rendu avec des paramétrages différents pour que l'application soit directement testable.

5. Conclusion

Ce projet nous a appris beaucoup de choses, que ce soit sur le langage C en lui-même ou sur la façon de travailler à deux sur un même programme. Au début c'était un peu difficile de s'organiser et de se répartir le code sans se marcher dessus, mais on a trouvé un rythme qui fonctionnait bien grâce à Git et Discord.

On est globalement satisfaits du résultat : l'application fonctionne, elle est robuste aux mauvaises saisies et elle répond à toutes les exigences du cahier des charges. On a même pu ajouter la gestion par catégories en bonus.